Sign-to-Speech: A Privacy-Preserving Decentralized Federated Learning Approach for American Sign Language Recognition

Truong Thien Nhan
ID: 10423085
Vietnamese-German University

Nguyen Quoc Khang
ID: 10423055
Vietnamese-German University

Nguyen Ho Nguyen
ID: 10423126
Vietnamese-German University

Nguyen Thien Phuc
ID: 10423164
Vietnamese-German University

Nguyen Thanh Danh
ID: 10423021
Vietnamese-German University

***Abstract*—This project implements a Decentralized Federated Learning (DFL) system for American Sign Language (ASL) sign to speech including text and audio system. Unlike traditional machine learning, which requires centralized data collection, the proposed approach allows five distributed nodes to train collaboratively while keeping all image data locally on each device. Each node trains on a different non-IID subset of ASL classes and exchanges only model weights through an asynchronous gossip protocol. The system improves privacy by preventing raw data sharing, increases fault tolerance by avoiding dependence on a central server, and supports horizontal scalability with minimal architectural changes. Experimental results demonstrate that decentralized federated learning can effectively perform ASL classification while preserving data privacy in distributed environments.**

***Index Terms*—Decentralized federated learning, peer-to-peer computing, gossip protocols, sign language recognition, MobileNetV2, transfer learning, edge computing, assistive technology.**

I. Introduction

Sign Language plays an important role in helping deaf and hard-of-hearing individuals communicate effectively in daily life. However, learning and understanding ASL through traditional methods can often be difficult, time- consuming, and less engaging for many users. With the advancement of artificial intelligence, computer vision, and machine learning technologies, Sign-to-Speak systems have become a promising solution for translating ASL gestures into text and speech in real time. Despite these advancements, many existing ASL recogni- tion systems rely on centralized machine learning approaches that require users to upload sensitive visual data to remote servers. This creates concerns regarding privacy, security, and data ownership. In addition, centralized systems may face scalability limitations and dependency on a single server infrastructure. To address these challenges, this project proposes a Sign-to- Speak system integrated with decentralized federated learning. The proposed framework allows users to train models locally on their own devices while only sharing model updates instead of raw data. Through a decentralized peer-to-peer architecture, the system enhances privacy protection while enabling collab- orative learning among multiple participants. The proposed system captures ASL gestures using webcams or mobile cameras and applies deep learning techniques to recognize signs and convert them into text and speech outputs. By combining ASL recognition with decentralized federated learning, this project aims to provide a more secure, scalable, and effective platform for communication, learning, and re- search applications.

II. Definition and Explaination

A. *Decentralized Federated Learning (DFL)*

- Decentralized Federated Learning (DFL) is a machine learning method where multiple devices work together to train a shared model without sending their private data to a central server or relying on a central coordinator.
- How Decentralized Federated Learning (DFL) works:
  - No Central Server: Nodes act as equal peers in a mesh network and talk directly to neighbors.
  - Data Stays Private: Nodes only train on their local images. Images are never sent over the network.
  - Weight Sharing (Gossip): Nodes convert their trained model weights into raw bytes and send them to neighbors using gRPC.
  - Weighted Averaging (FedAvg): Nodes average their own weights with received peer weights (giving more weight to nodes with more data) and update their models.
  - Reliability: If one node goes offline, the other nodes keep training and sharing weights without stopping.

B. *Centralized Federated Learning (CFL)*

- CFL is a machine learning method where multiple devices train a shared model under the command of one

central coordinator server, without sharing their private data.

- How Centralized Federated Learning (CFL) Works
  - Central Coordinator Server: A single master server manages all client nodes, distributes the model, and performs all aggregations.
  - Data Stays Private: Nodes only train on their local images. Images are never sent over the network.
  - Weight Synchronization: Nodes upload their trained model weights to the central server and download the updated global weights back.
  - Central Aggregation (FedAvg): The central server collects weights from all nodes, calculates their average, and updates the main global model.
  - Vulnerability: If the central server goes offline, the entire training process stops and the system fails.

III. Problem Statement

Here are some of the problem that we have to face while making this project

- Dataset Insufficient: At first, our dataset is too small in order for the computer to learn. As a result, generalization, bias, and overfitting (where the model memorizes the data instead of learning to detect real) appear in our project
- Dataset Overload: Another problem that we face is that we try too load many ASL picture to the dataset and train it. It causes us fragmented and disparate source, redundant data and poor data governance
- Background Distribution: One of our weakness is the backgorund. Because we have filter from several dataset with different background so it was really difficult for our computer to train and learn with high accuracy
- System Overload: Crashing is one of the trouble that we faced while making the AI, due to large dataset and epoch, our computer sometimes have some malfunction which delay our progress

IV. Teamwork

TABLE I: Task Distribution

No	Task Description	Responsible Member(s)
1	P2P and Decentralized Orchestrator	Truong Thien Nhan
2	Model and Local Training	Nguyen Thanh Danh
3	Partitioning and Pre-processing	Truong Thien Nhan, Nguyen Quoc Khang, Nguyen Ho Nguyen (main)
4	Network and Simulation	Nguyen Quoc Khang
5	Translator and UI	Nguyen Thien Phuc

A. P2P and Decentralized Orchestrator

Truong Thien Nhan designed and implemented the gRPC communication layer and the decentralized P2P training orchestration loop in `node.py`. In a decentralized federated learning network, coordinating nodes is highly challenging due to the absence of a central coordinator, varying node speeds, and the concurrency of local model fitting and peer model synchronization. To solve these challenges, Truong Thien Nhan designed a robust gRPC protocol, a high-performance weight serialization interface, and a thread-safe FIFO buffer with a strict memory-eviction policy.

B. Model and Local Training

This section will introduce the fundamental of machine learning architecture, local training framework that implemented in our decentralized federated project. Making sure user privacy, raw training images remain securely on their host nodes, the place which convolution networks perform localized training. MobileNetV2 is the backbone of our approach that utilized the transfer learning. As a result, enhanced with a multi-layered classification head designed to resolve highly identical hand configuration. The training loop is sercured, restricted using in-graph image augmentations, early stopping strategies, label-smoothed classification smooth. The model parameter are converted into raw float32 byte arrays and asynchronously shared among nodes using gRPC gossip protocol, allow the cluster co-train a robust model collaboratively.

- Dataset: https://data.mendeley.com/datasets/j4y5w2c8w9/1/files/5a69be0c-1f83-45b2-b7d9-6710b585cacc
- Library usage:
  - **Deep Learning and Machine Learning**
    - TensorFlow: Design, compile, train, and run predictions on neural networks.
  - **Numerical and Mathematical Computation**
    - NumPy: Provides high-performance multidimensional array operations, matrix computations, and raw weight serialization/deserialization.
  - **Distributed Network and Communication Protocol**
    - gRPC: Responsible for low-latency, secure P2P transport of model weights between Docker nodes.
    - Protocol Buffers (`dfl_service_pb2`, `dfl_service_pb2_grpc`): Handles data serialization schema contracts.
  - **Multithreading and Memory Safety**
    - threading: Uses `threading.Lock` to prevent conflicts when the gRPC server thread writes weights and the main thread reads them for FedAvg aggregation.
    - gc.collect(): Frees unused memory from Keras models and arrays.
  - **Data Parsing and System Utilities**
    - Pandas (`pandas`): Reads CSV files.
    - json (`json`): Reads and writes configuration metadata.
    - os & sys (`os`, `sys`): Filesystem and environment utilities.

V. System Implementation Pipeline

A. Step 1: Build the Baseline Model ($model_mnist.py$)

First, we construct a simple CNN to verify the end-to-end training pipeline before using MobileNetV2.

```
        model = models.Sequential([
            layers.Input(shape=(28, 28,
1)),
            layers.Conv2D(32, (3,3),
activation='relu'),
            layers.MaxPooling2D(2, 2),
            layers.Conv2D(64, (3,3),
activation='relu'),
            layers.MaxPooling2D(2, 2),
            layers.Flatten(),
            layers.Dense(128,
activation='relu'),
            layers.Dense(num_classes,
activation='softmax')
        ])
        model.compile(
            optimizer='adam',

loss='categorical_crossentropy',
            metrics=['accuracy']
        )
```

Goal: Confirm the training pipeline works end-to-end before committing to MobileNetV2.

B. Step 2: Test the Baseline Model ($train_mnist.py$)

```
        train =
pd.read_csv("data/mnist/sign_mnist_train.csv")
        X_train = train.iloc[:,
1:].values.reshape(-1, 28, 28, 1) /
255.0
        y_train =
to_categorical(train.iloc[:, 0].values)
        model =
build_model(num_classes=y_train.shape[1])
        model.fit(X_train, y_train,
epochs=5)
        loss, acc =
model.evaluate(X_test, y_test)
        print("MNIST Accuracy:", acc)
        json.dump(get_weights(model),
open("weights_mnist.json", "w"))
```

Goal: Obtain baseline accuracy and save model weights for integration testing.

C. Step 3: Weight Extraction Utility (utils.py)

```
        def get_weights(model):
            return [w.tolist() for w in
model.get_weights()]
        def load_weights(model, weights):
            import numpy as np

model.set_weights([np.array(w) for w
in weights])
```

Goal: Enable serialization and deserialization of model weights.

D. Step 4: Build the Advanced Model ($model_image.py$)

```
            base_model = MobileNetV2(
                input_shape=(160,160,3),
                include_top=False,
                weights='imagenet'
            )
            base_model.trainable = False




        x = layers.Rescaling(1.0/127.5,
offset=-1.0)(inputs)




        x =
layers.GlobalAveragePooling2D()(x)
        x = layers.BatchNormalization()(x)
        x = layers.Dense(512,
activation='relu')(x)
        x = layers.Dropout(0.4)(x)
        x = layers.Dense(128,
activation='relu')(x)
        x = layers.Dropout(0.3)(x)
        outputs =
layers.Dense(num_classes,
activation='softmax')(x)




        ft_schedule = CosineDecayRestarts(
            initial_learning_rate=5e-4,
            first_decay_steps=2500
        )
        model.compile(
            optimizer=Adam(lr_schedule),
            loss=CategoricalCrossentropy(
                label_smoothing=0.1
            ),
            metrics=['accuracy']
        )
```

Goal: Build a production-grade ASL classification model.

E. Step 5: Fine-Tuning Strategy

```
        def fine_tune_model(model,
num_unfreeze=30):
            base_model.trainable = True
            for layer in
base_model.layers[:-num_unfreeze]:
                layer.trainable = False
            for layer in
base_model.layers:
                if isinstance(layer,
BatchNormalization):
                    layer.trainable =
False
            model.compile(

optimizer=Adam(CosineDecayRestarts(

initial_learning_rate=1e-5,
```

```
first_decay_steps=2500,
                alpha=1e-7
            )),

loss=CategoricalCrossentropy(label_smoothing=0.1),
            metrics=['accuracy']
        )
```

Goal: Adapt pretrained backbone to ASL domain.

F. Step 6: Full Model Training

```
    history1 = model.fit(
        train_ds_aug,
        epochs=20,
        validation_data=val_ds_cached,

callbacks=[EarlyStopping(patience=5),

ModelCheckpoint("phase1_best.keras")]
    )
    loss, acc =
model.evaluate(val_ds_cached)
    print(acc)
    if acc >= 0.70:
        model =
fine_tune_model(model, num_unfreeze=30)

        history2 = model.fit(
            train_ds_aug,
            epochs=25,

validation_data=val_ds_cached
        )
    json.dump([w.tolist() for w in
model.get_weights()],

open("weights_image.json", "w"))
```

Goal: Establish accuracy ceiling for federated learning comparison.

G. Step 7: Runtime Model Adapter (model.py)

```
    def build_model(num_classes=26):
        if MODEL_TYPE == "mnist":
            from model_mnist import
build_model as _build
            return _build()
        return _build_image_model(
            input_shape=(160,160,3),
            num_classes=num_classes
        )


    def serialize_weights(model):
        flat = np.concatenate(
            [w.flatten() for w in
model.get_weights()]
        ).astype(np.float32)
        return flat.tobytes()

    def deserialize_weights(model,
data):
        flat = np.frombuffer(
            data,
            dtype=np.float32
        ).copy()
        shapes = [
            w.shape
            for w in
model.get_weights()
        ]
        weights, offset = [], 0
        for shape in shapes:
            size = int(
                np.prod(shape)
            )
            weights.append(
                flat[
                    offset:offset +
size
                ].reshape(shape)
            )
            offset += size
        model.set_weights(weights)
```

Goal: Enable gRPC-based federated weight exchange.

H. Step 8: Local Training in Node (node.py)

```
    data_augmentation =
tf.keras.Sequential([

tf.keras.layers.RandomRotation(0.15),

tf.keras.layers.RandomTranslation(0.10,
0.10),

tf.keras.layers.RandomZoom(0.15),

tf.keras.layers.RandomBrightness(0.20),

tf.keras.layers.RandomContrast(0.20),
    ])
    def train_local(model,
train_data, val_data, epochs=5):
        early_stop =
tf.keras.callbacks.EarlyStopping(
            monitor='val_accuracy',
            patience=3,
            restore_best_weights=True
        )

        model.fit(
            train_data,
            epochs=epochs,
            validation_data=val_data,
            callbacks=[early_stop]
        )

        return
model.evaluate(val_data)
```

Goal: Train model locally per federated learning node.

I. Step 9: Evaluation and Round Tracking

```
    loss, acc = train_local(model,
train_data, val_data)
    round_history.append((round_num,
loss, acc))
    print(f"Round {round_num} |
loss={loss:.4f} | acc={acc:.2f}")
    if global_val_data:
        g_loss, g_acc =
model.evaluate(global_val_data)
        print(g_acc)
```

Goal: Track federated learning performance across rounds.

J. Step 10: Checkpoint Saving

```
    def save_checkpoint(model,
round_num, round_history):

np.save("checkpoints/weights.npy",

np.array(model.get_weights(),
dtype=object))

        json.dump({
            "round_num": round_num,
            "round_history":
round_history
        },
open("checkpoints/checkpoint.json",
"w"))
```

Goal: Ensure fault tolerance and resume capability.

- Training Result

```
node1-1  | [Node 1] ===== TRAINING COMPLETE (30 rounds) =====
node1-1  | [Node 1]  Round   1 | loss=1.0477 | acc=89.71%
node1-1  | [Node 1]  Round   2 | loss=0.9648 | acc=92.26%
node1-1  | [Node 1]  Round   3 | loss=0.9693 | acc=91.92%
node1-1  | [Node 1]  Round   4 | loss=0.9252 | acc=93.25%
node1-1  | [Node 1]  Round   5 | loss=0.8999 | acc=95.02%
node1-1  | [Node 1]  Round   6 | loss=0.8834 | acc=96.46%
node1-1  | [Node 1]  Round   7 | loss=0.8495 | acc=96.79%
node1-1  | [Node 1]  Round   8 | loss=0.8538 | acc=96.57%
node1-1  | [Node 1]  Round   9 | loss=0.8308 | acc=97.23%
node1-1  | [Node 1]  Round  10 | loss=0.8666 | acc=97.01%
node1-1  | [Node 1]  Round  11 | loss=0.8333 | acc=98.67%
node1-1  | [Node 1]  Round  12 | loss=0.8190 | acc=98.78%
node1-1  | [Node 1]  Round  13 | loss=0.8222 | acc=98.12%
node1-1  | [Node 1]  Round  14 | loss=0.8240 | acc=98.34%
node1-1  | [Node 1]  Round  15 | loss=0.8016 | acc=98.45%
node1-1  | [Node 1]  Round  16 | loss=0.7916 | acc=98.45%
node1-1  | [Node 1]  Round  17 | loss=0.7833 | acc=99.34%
node1-1  | [Node 1]  Round  18 | loss=0.7944 | acc=99.00%
node1-1  | [Node 1]  Round  19 | loss=0.7676 | acc=99.56%
node1-1  | [Node 1]  Round  20 | loss=0.7831 | acc=99.45%
node1-1  | [Node 1]  Round  21 | loss=0.7799 | acc=99.45%
node1-1  | [Node 1]  Round  22 | loss=0.7897 | acc=99.12%
node1-1  | [Node 1]  Round  23 | loss=0.7782 | acc=99.45%
node1-1  | [Node 1]  Round  24 | loss=0.7624 | acc=99.78%
node1-1  | [Node 1]  Round  25 | loss=0.7664 | acc=99.45%
node1-1  | [Node 1]  Round  26 | loss=0.7747 | acc=99.67%
node1-1  | [Node 1]  Round  27 | loss=0.7585 | acc=100.00% ← PEAK
node1-1  | [Node 1]  Round  28 | loss=0.7707 | acc=99.34%
node1-1  | [Node 1]  Round  29 | loss=0.7655 | acc=99.56%
node1-1  | [Node 1]  Round  30 | loss=0.7489 | acc=100.00%
node1-1  | [Node 1] Model exported.
node1-1  | [Node 1] Peak val_acc=100.00% at round 27
node1-1 exited with code 0
```

Fig. 1: Node 1

```
node2-1  | [Node 2] ===== TRAINING COMPLETE (30 rounds) =====
node2-1  | [Node 2]  Round   1 | loss=0.9679 | acc=92.42%
node2-1  | [Node 2]  Round   2 | loss=0.9072 | acc=94.77%
node2-1  | [Node 2]  Round   3 | loss=0.8907 | acc=95.59%
node2-1  | [Node 2]  Round   4 | loss=0.8460 | acc=96.72%
node2-1  | [Node 2]  Round   5 | loss=0.8438 | acc=96.00%
node2-1  | [Node 2]  Round   6 | loss=0.8328 | acc=97.23%
node2-1  | [Node 2]  Round   7 | loss=0.8400 | acc=96.93%
node2-1  | [Node 2]  Round   8 | loss=0.8162 | acc=97.64%
node2-1  | [Node 2]  Round   9 | loss=0.8184 | acc=97.44%
node2-1  | [Node 2]  Round  10 | loss=0.8173 | acc=97.85%
node2-1  | [Node 2]  Round  11 | loss=0.8084 | acc=97.75%
node2-1  | [Node 2]  Round  12 | loss=0.8074 | acc=98.05%
node2-1  | [Node 2]  Round  13 | loss=0.7945 | acc=98.16%
node2-1  | [Node 2]  Round  14 | loss=0.7859 | acc=98.26%
node2-1  | [Node 2]  Round  15 | loss=0.7749 | acc=98.98%
node2-1  | [Node 2]  Round  16 | loss=0.7571 | acc=99.69%
node2-1  | [Node 2]  Round  17 | loss=0.7582 | acc=98.87%
node2-1  | [Node 2]  Round  18 | loss=0.7615 | acc=99.08%
node2-1  | [Node 2]  Round  19 | loss=0.7511 | acc=99.59%
node2-1  | [Node 2]  Round  20 | loss=0.7414 | acc=99.39%
node2-1  | [Node 2]  Round  21 | loss=0.7502 | acc=99.08%
node2-1  | [Node 2]  Round  22 | loss=0.7501 | acc=99.08%
node2-1  | [Node 2]  Round  23 | loss=0.7579 | acc=98.67%
node2-1  | [Node 2]  Round  24 | loss=0.7401 | acc=99.69%
node2-1  | [Node 2]  Round  25 | loss=0.7418 | acc=99.49%
node2-1  | [Node 2]  Round  26 | loss=0.7400 | acc=99.59%
node2-1  | [Node 2]  Round  27 | loss=0.7357 | acc=99.18%
node2-1  | [Node 2]  Round  28 | loss=0.7489 | acc=99.90% ← PEAK
node2-1  | [Node 2]  Round  29 | loss=0.7424 | acc=99.69%
node2-1  | [Node 2]  Round  30 | loss=0.7339 | acc=99.80%
node2-1  | [Node 2] Model exported.
node2-1  | [Node 2] Peak val_acc=99.90% at round 28
node2-1 exited with code 0
```

Fig. 2: Node 2

```
node3-1  | [Node 3] ===== TRAINING COMPLETE (30 rounds) =====
node3-1  | [Node 3]  Round   1 | loss=0.9950 | acc=91.18%
node3-1  | [Node 3]  Round   2 | loss=0.9568 | acc=92.88%
node3-1  | [Node 3]  Round   3 | loss=1.0273 | acc=89.01%
node3-1  | [Node 3]  Round   4 | loss=0.9017 | acc=94.74%
node3-1  | [Node 3]  Round   5 | loss=0.9049 | acc=95.20%
node3-1  | [Node 3]  Round   6 | loss=0.8886 | acc=94.58%
node3-1  | [Node 3]  Round   7 | loss=0.8696 | acc=95.36%
node3-1  | [Node 3]  Round   8 | loss=0.8382 | acc=96.28%
node3-1  | [Node 3]  Round   9 | loss=0.8576 | acc=96.75%
node3-1  | [Node 3]  Round  10 | loss=0.8317 | acc=96.90%
node3-1  | [Node 3]  Round  11 | loss=0.8272 | acc=96.90%
node3-1  | [Node 3]  Round  12 | loss=0.8348 | acc=96.90%
node3-1  | [Node 3]  Round  13 | loss=0.8039 | acc=98.14%
node3-1  | [Node 3]  Round  14 | loss=0.7997 | acc=97.68%
node3-1  | [Node 3]  Round  15 | loss=0.7803 | acc=98.92%
node3-1  | [Node 3]  Round  16 | loss=0.7803 | acc=99.23%
node3-1  | [Node 3]  Round  17 | loss=0.7737 | acc=98.92%
node3-1  | [Node 3]  Round  18 | loss=0.7860 | acc=99.38%
node3-1  | [Node 3]  Round  19 | loss=0.7764 | acc=98.76%
node3-1  | [Node 3]  Round  20 | loss=0.7774 | acc=98.61%
node3-1  | [Node 3]  Round  21 | loss=0.7677 | acc=99.38%
node3-1  | [Node 3]  Round  22 | loss=0.7820 | acc=99.54%
node3-1  | [Node 3]  Round  23 | loss=0.7606 | acc=99.69%
node3-1  | [Node 3]  Round  24 | loss=0.7584 | acc=99.69%
node3-1  | [Node 3]  Round  25 | loss=0.7476 | acc=99.69%
node3-1  | [Node 3]  Round  26 | loss=0.7479 | acc=99.54%
node3-1  | [Node 3]  Round  27 | loss=0.7541 | acc=99.69%
node3-1  | [Node 3]  Round  28 | loss=0.7621 | acc=100.00% ← PEAK
node3-1  | [Node 3]  Round  29 | loss=0.7449 | acc=99.85%
node3-1  | [Node 3]  Round  30 | loss=0.7521 | acc=99.69%
node3-1  | [Node 3] Model exported.
node3-1  | [Node 3] Peak val_acc=100.00% at round 28
node3-1 exited with code 0
```

Fig. 3: Node 3

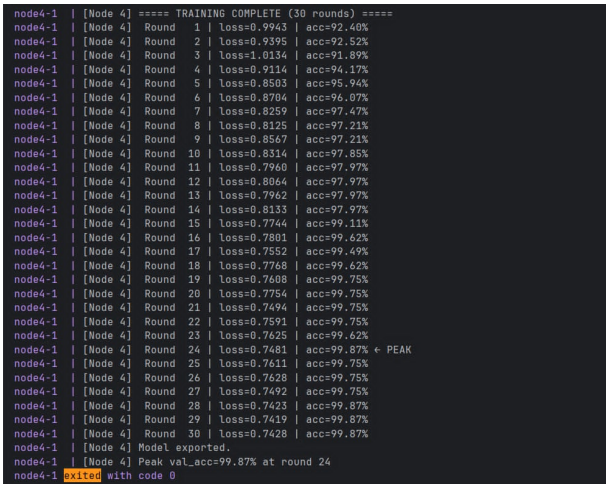

Fig. 4: Node 4

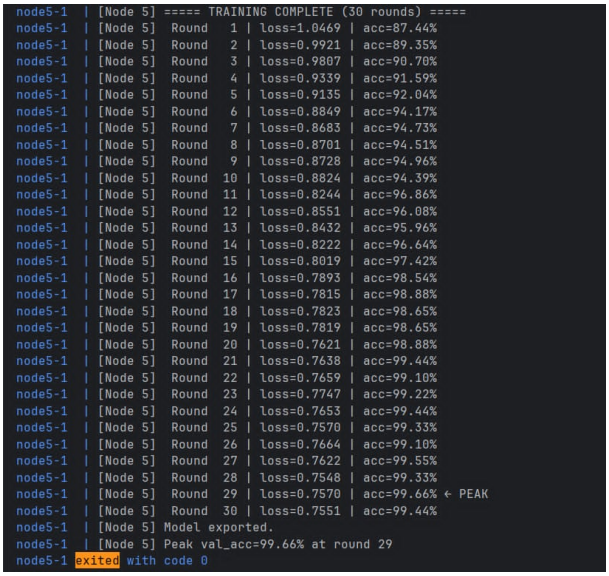

Fig. 5: Training result of Node 5

K. Partitioning and Pre-processing

1) Image Resizing: Before data sharding, all raw ASL images are standardised to a uniform resolution using `resize.py`. The script walks every class folder in the source directory, opens each image with the Pillow library, converts it to RGB, and downscales it to 160×160 pixels using the LANCZOS resampling filter, which gives the best quality for downsampling. The class-folder structure is preserved in the output directory (`resized_dataset/`):

```
img =
Image.open(img_path).convert("RGB")
img = img.resize((160, 160),
Image.LANCZOS)
img.save(save_path)
```

2) Data Sharding: `partition.py` divides the pre-processed dataset into five per-node shards that simulate a realistic non-IID (non-identically distributed) federated environment. Each node receives a different proportion of each ASL class, reflecting the heterogeneous data distributions found in real-world deployments.

Dirichlet-based Non-IID Allocation. For each class c, the proportion of images assigned to the $N = 5$ nodes is drawn from a Dirichlet distribution:

$$\mathbf{p}^{(c)} \sim \text{Dir}(\alpha \cdot \mathbf{1}_N), \quad \alpha = 0.5 \tag{1}$$

The concentration parameter α controls the degree of heterogeneity: $\alpha = 0.5$ produces a strongly non-IID split where each node specialises in a subset of classes, while $\alpha \to \infty$ approaches a uniform IID distribution. The image count assigned to node i for class c is:

$$n_i^{(c)} = \left\lfloor p_i^{(c)} \cdot |C^{(c)}| \right\rfloor \tag{2}$$

where $|C^{(c)}|$ is the total number of images in class c.

```
proportions =
np.random.dirichlet([alpha] *
num_nodes)
raw_counts = (proportions *
n_images).astype(int)
```

Minimum-Floor Correction. A floor of `MIN_PER_CLASS = 10` is enforced so that every node always receives at least 10 images per class, preventing *class starvation* where a node has no samples for certain gestures and cannot learn them. If the Dirichlet draw falls below the floor, the count is raised to 10. If the total then exceeds available images, excess is removed from the largest allocations first while the floor is preserved:

```
for i in range(num_nodes):
    if raw_counts[i] < MIN_PER_CLASS:
        raw_counts[i] = MIN_PER_CLASS
```

Train/Validation Split. Within each node's allocation, 80% of images are assigned to training and 20% to local validation. The split is applied per class to maintain class balance within each partition:

```
split_idx    = max(1,
int(len(node_images) * 0.8))
train_images = node_images[:split_idx]
val_images   = node_images[split_idx:]
```

Metadata Output. A `metadata.json` file is written to each node's root directory containing the node ID, total training count, validation count, and per-class label distribution. Each node's local trainer reads this file at startup to verify data integrity. The final shard layout is:

```
data_shards/
  node1/ train/{A..Z}/ val/{A..Z}/
metadata.json
```

```
    ...
    node5/ train/{A..Z}/ val/{A..Z}/
metadata.json
```

L. Network and Simulation

The network and simulation component of this project is implemented using Docker and Docker Compose. Instead of requiring five separate physical machines, the system simulates a decentralized federated learning environment by running five independent containers on a single host machine. Each container represents one learning node in the decentralized network.

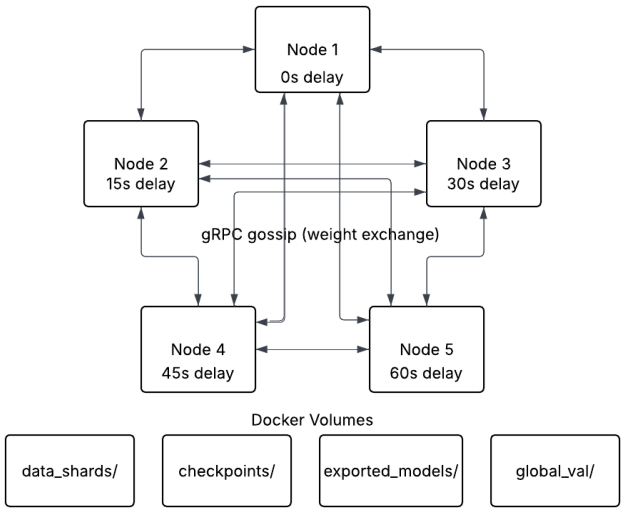

Fig. 6: Docker Compose network topology: five containerised nodes exchanging model weights via gRPC gossip over a virtual bridge network, with per-node and shared volume mounts.

The simulation consists of five services: `node1`, `node2`, `node3`, `node4`, and `node5`. Each service is built from the same Docker image, but each node is assigned a different configuration through environment variables. The most important variable is `NODE_ID`, which allows the application to identify which simulated node is running inside the container.

Each node also receives its own local data directory through Docker volume mounting. For example, `node1` uses the directory `./data_shards/node1`, while `node2` uses `./data_shards/node2`. Inside the container, these directories are mounted to `/app/data`. This design ensures that each node trains only on its assigned local dataset, similar to how different users or devices would each own their own private data in a real decentralized federated learning system.

The Docker Compose configuration also mounts separate checkpoint folders for each node. These checkpoint folders are used to store training progress and model states during the experiment. This makes the simulation more fault tolerant because nodes can continue from saved progress instead of starting again from the beginning after interruption.

A shared `exported_models` directory is mounted into all containers so that trained or exported models can be collected in one common location. In addition, a read-only global validation directory is mounted using `./global_val:/app/global_val:ro`. This allows nodes to evaluate model performance on a common validation set while preventing accidental modification of the validation data.

TABLE II: Docker Compose Environment Variables

Variable	Value	Purpose
`LOCAL_EPOCHS`	5	Local training epochs per round
`MAX_ROUNDS`	30	Maximum federated rounds
`NUM_CLASSES`	26	ASL alphabet classes
`MODEL_TYPE`	image	Selects the MobileNetV2 pipeline
`BATCH_SIZE`	32	Images processed per training step

To reduce resource pressure during initialisation, nodes are started with staggered delays via Docker Compose `depends_on` conditions:

1) **Node 1** — starts immediately (0 s)
2) **Node 2** — delayed 15 s
3) **Node 3** — delayed 30 s
4) **Node 4** — delayed 45 s
5) **Node 5** — delayed 60 s

The Dockerfile defines the runtime environment used by every node. It installs Python, required system libraries, and all Python dependencies from `requirements.txt`. The working directory is set to `/app`, and the project files are copied into the container. The default command runs `python app/node.py`, which starts the node application. This ensures that all five simulated nodes run under the same software environment, improving reproducibility and reducing configuration errors.

In general, the network simulation allows the project to test decentralized federated learning behavior in a controlled and repeatable way. Docker containers provide isolation between nodes, Docker volumes provide separate local datasets and checkpoints, and Docker Compose allows the whole multi-node system to be started with a single command. This makes it possible to evaluate the decentralized learning system without needing multiple physical computers.

M. Translator (Audio) and UI

Nguyen Thien Phuc built the interactive Streamlit user interface for live testing. He developed the MediaPipe-based hand crop module, designed the convex-hull background mask, and integrated the pyttsx3 Text-to-Speech translation engine. He also added temporal majority voting to stabilize real-time webcam predictions. To be more interactive, the program goes with a user interface

and an audio generator. The core application of this program is applied with Streamlit, OpenCV, MediaPipe, and Pyttsx3 extensions. With these implementations, the interface is able to handle both user experiences and backend communication.

*1) **Preprocessing**:* To ensure the consistency of images before sending to the prediction system, a setup for the image is required. The UI handles this modification.

- **Hand cropping:** After the camera was opened, the UI used MediaPipe to detect where the hand is and find its bounding box, with the default bounding box of 15%, to avoid size mismatch. The cropped area is then padded with black borders to the size of 160×160 pixels to fit the prediction model.
- **Contrast Equalization (CLAHE):** The UI applies CLAHE (Contrast Limited Adaptive Histogram Equalization) of CV2 to the Y (luma) channel in the YUV color space before sending the image to MediaPipe. This act enhances the contrast of the picture for better accuracy in difficult lighting or shadows.

*2) **Inferencing**:* After being trained through the decentralized network, the MobileNetV2 model, which is the backend, takes a 160×160 image as input from the UI and returns a letter.

- **Confidence Threshold Adjustment:** Because the training process uses label smoothing, the model's output probabilities are better corrected. Therefore, the minimum confidence threshold is preset at 0.55, and if any prediction is lower than this, the system will give a warning.
- **Temporal Smoothing:** In live camera mode, to enhance the accuracy of predicting letters, a 7-frame sliding window `deque` is used. Together with the majority voting, confusing of the prediction can be eliminated. Along with it, the model also returns the top 5 predictions and shows the data from the most recent frame of the winning letter.

*3) **Speeching**:* To fulfill the term Sign-to-Speech, the last part of UI is the speaker. Once a letter is predicted, the Text-to-Speech library is used, and the letter is logged. Streamlit works by reloading the whole template every time the user interacts, `rerun`, with it. And this can cause thread-safety issues with the audio engine. To fix this, the code created a new thread and a new TTS engine for every audio process. Each time the system speaks a letter, it starts a new, separate background thread and a completely new TTS engine. This makes sure the sound plays instantly and prevents the UI from crashing.

VI. Implementation Details

This section explains how we actually built the Decentralized Federated Learning (DFL) Sign-to-Speech system. It shows how we turned complex theories into a real, working process. This process is designed to be fast and save memory, allowing it to run smoothly on small devices (which we simulated using Docker). In this section, it describes the core intelligence of the nodes, focusing on the local neural network architecture, regularization schedules, and weight formatting protocols.

A. Local Model Architecture and Transfer Learning Base

To make sure the system runs smoothly on small devices without powerful graphics cards (GPUs), we use a highly efficient, lightweight AI model. We chose MobileNetV2 as our foundation because its unique design drastically cuts down on calculations and saves memory.

The data and the network are set up in two steps, Image Size and Pre-trained Layers. First, Image Size, every device is standardly set up to process color images at 160×160 pixels (with 3 color channels). This is slightly larger than the usual 128×128 size, which helps the system clearly see fine details like finger shapes. Second, Pre-trained Layers, we "freeze" (lock) the core layers of MobileNetV2 at the start. Because these layers were already trained on a massive dataset (ImageNet), they can already recognize basic visual elements like edges, lines, and textures perfectly.

To connect these visual features to the 26 different American Sign Language (ASL) letters and numbers, we add a custom "classification head" (the final layers of the model) to the end of the network. The layers of this classification head are structured as follows:

1) **Global Average Pooling 2D:** Collapses the spatial dimensions of the feature maps into a unified feature vector, preventing spatial overfitting.
2) **Dense Feature Layer:** A fully-connected layer containing 512 neurons utilizing the Rectified Linear Unit (ReLU) activation function to introduce non-linear decision boundaries.
3) **Dropout Regularization (Rate = 0.4):** Randomly deactivates 40% of the node activations during forward passes to force feature redundancy and eliminate co-adaptation.
4) **Dense Projection Layer:** A secondary fully-connected layer containing 128 neurons with ReLU activation to compress the feature representation.
5) **Dropout Regularization (Rate = 0.3):** A secondary regularizer providing further protection against local overfitting.
6) **Softmax Output Layer:** A dense projection with 26 units generating a normalized probability distribution representing the classification confidence scores across the ASL label mapping.

B. Regularization and Local Optimization Pipeline

Training models independently on separated data (non-IID shards) can cause big problems. The models might learn completely different things and become too confident in the wrong answers. To make the training more

stable before sharing updates with the whole network, we use two special techniques during the local training process:

1) Label Smoothing: Standard training forces a model to be 100% sure (1.0) about the correct answers. On difficult, separated data, this causes difficulty in learning. We fix this by labeling. We adjust the target answer y by adding a small amount of random noise ($\epsilon = 0.1$):

$$y'_k = y_k(1 - \epsilon) + \frac{\epsilon}{K} \tag{3}$$

Here, $K = 26$ is the total number of classes. This stops the Softmax layer from giving overconfident probabilities. Because of this, it changes how the system makes predictions. When running the code in `app_tester.py`, we lower the confidence limit from 0.60 to 0.55. The model's output percentage are now much more realistic than before.

2) Cosine Decay Learning Rate Scheduling: If we use a fixed learning speed (learning rate), the model might get stuck and jump around the best answer. Instead, the local trainer uses a Cosine Decay plan. The learning rate η_t smoothly gets smaller over time based on this formula:

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min}) \left(1 + \cos\left(\frac{T_{cur}}{T_{max}}\pi\right)\right) \tag{4}$$

It starts at $\eta_{max} = 10^{-3}$ and drops down to a minimum of 10^{-5} by the end of the training rounds. This helps the model reach the final result smoothly and accurately.

C. Weight Serialization and Network Payload Formatting

A main problem in peer-to-peer decentralized machine learning is the time and network power it takes to send complex model parameters to one another. Sending heavy Keras models or large TensorFlow data directly over the network causes delays. It also can cause data errors between different computers (nodes).

To fix this, `model.py` uses a strong system to pack and unpack data (serialization and deserialization). Before sending data via gRPC, the local node gets the weights from the model's layers. These large data shapes are flattened and combined into one 1D list. Then, they are converted into a 32-bit format (`np.float32`).

Next, the array is packed into a basic binary format (raw bytes). This method shrinks the model's parameters into a single package of about 9.2 MB. This small size reduces heavy work for the network inside the Docker setup and allows data sharing using gRPC. The code for packing the data looks like this:

```
def serialize_weights(model) -> bytes:
    # Extract, flatten, and
concatenate tensors
    flat = np.concatenate([
        w.flatten() for w in
model.get_weights()
    ]).astype(np.float32)
    return flat.tobytes()
```

When a node receives the data from a peer, it reverses the process. It takes the incoming raw bytes and safely copies them into memory to avoid computer errors. Then, it changes the list of numbers back into the correct shapes needed for each layer of the MobileNetV2 model:

```
     def deserialize_weights(model,
data: bytes) -> None:
    flat = np.frombuffer(data,
dtype=np.float32).copy()
    shapes = [w.shape for w in
model.get_weights()]
    weights = []
    curr = 0
    for shape in shapes:
        size = np.prod(shape)
        w_slice = flat[curr:curr +
size].reshape(shape)
        weights.append(w_slice)
        curr += size
    model.set_weights(weights)
```

D. Asynchronous Peer-to-Peer gRPC Orchestration Loop

The decentralized network does not need a central server. Instead, it uses a fast gRPC communication method where nodes share updates at different times. Every computer (node) in the network acts as both a gRPC Server (to listen for updates) and a gRPC Client (to send its own data).

To send the packed 1D binary array, which is about 9.2 MB per message, we must increase the normal gRPC message size limits. Both the sending and receiving channels are using special settings to stop messages from being disrupted:

```
grpc_options = [
    ("grpc.max_send_message_length",
  32 * 1024 * 1024),

("grpc.max_receive_message_length", 32
* 1024 * 1024),
]
```

This change sets a maximum message size of 32 MB. This makes sure there is enough space to send the model data over the network. The whole training process runs for up to $T_{max} = 30$ global rounds. Each communication round follows a strict time limit without pausing, using a fixed sharing time ($G_{interval} = 30$ seconds):

1) **Synchronization Phase:** The node remains idle for 30 seconds, during which its background gRPC server asynchronously accepts and accumulates incoming weight payloads from any active topological neighbors.

2) **Consensus & Merge Phase:** The node isolates the received weights, executes local Federated Averaging (FedAvg) aggregation, and overwrites its active network layers with the updated parameters.
3) **Local Adaptation Phase:** The node trains the newly aggregated model against its highly unique local training data shard using the early stopping regularization loop.
4) **Gossip Phase:** The locally optimized weights are serialized and broadcast concurrently via unary gRPC streaming calls to all designated neighbor container addresses.

E. Thread-Safe FIFO Buffer Optimization and Local Aggregation

In a P2P network, computers (nodes) run on their own time. A big problem happens when fast nodes send too many model updates to a slow node at the same time. This can use up all the memory and cause the system to crash.

To stop this, `node.py` uses a safe memory system to manage the waiting list of updates. It uses a special lock (`threading.Lock`) to protect the data. This stops errors when different parts of the program try to add or remove data at the same time. Also, to control memory use, the list has a strict size limit. It removes the oldest data first (First-In, First-Out, or FIFO):

```
MAX_BUFFER_SIZE = 10
```

By limiting the list to 10 spaces, the maximum memory used is about 90 MB ($\approx 10 \times 9.2$ MB). This keeps small devices safe. If a new update arrives when the list is full, the system deletes the oldest update to leave space for the new one:

```
with buffer_lock:
    if len(received_buffer) <
MAX_BUFFER_SIZE:
        received_buffer.append((flat,
peer_n))
    else:
        received_buffer.pop(0) # FIFO
Eviction
        received_buffer.append((flat,
peer_n))
```

At the start of the sharing phase, the main program locks the list and makes a local copy of the data. Then, it clears the receiving list so it can accept new messages without stopping. The local weights (W_{local}) are mixed with the received peer weights ($W_{peer}^{(i)}$). This mixing uses the number of training samples to find the average:

$$W_{agg} = \frac{n_{local} W_{local} + \sum_{i=1}^{M} n_{peer}^{(i)} W_{peer}^{(i)}}{n_{local} + \sum_{i=1}^{M} n_{peer}^{(i)}} \tag{5}$$

where n_{local} and $n_{peer}^{(i)}$ are the exact numbers of training examples taken from each node's settings.

After combining the weights, the local training process uses a rule to stop early if needed. It has a patience setting ($P = 3$) that watches the local accuracy (`val_accuracy`). This setting does two important things: it gives the model time to recover after mixing different weights, and it stops the model from being overfitting.

F. Advanced Visual Preprocessing, Streaming UI, and Multi-Threaded Audio Speech Synthesis

The user application is built with the Streamlit framework to make the deep learning model easy for anyone to use. Before sending live video data to the model, the system processes it in two steps:

1) Contrast Limited Adaptive Histogram Equalization (CLAHE): To fix lighting differences from different user environments, the input image is changed into the YUV color space. CLAHE is used directly on the brightness (Y) channel with a limit of 2.0 and an 8×8 grid size. This improves local contrast without making background noise worse, helping the model see features clearly in dark or very bright rooms.

2) MediaPipe Landmark-Guided Square Slicing: The improved images go to a MediaPipe Hands system to find 21 specific hand joints. The positions of these joints are used to make a box around the hand. We add a 15% extra border (padding) so fingers do not get cut off when the hand moves. This area is cut out and resized to a 160×160 resolution using `cv2.INTER_AREA` to fit what the model needs. If no hand is found, the app cuts the center of the image instead so it keeps working.

To stop the predictions from jumping or flickering due to camera blur or noise during live video, the system uses a smooth voting method. The app remembers a short history of recent frames using a special double-ended queue (`deque`):

```
SMOOTHING_WINDOW = 7
```

The final prediction shown to the user is calculated using a majority vote across the last 7 sequential frames, which stabilizes the output text stream.

When the model is very sure about a new sign language word (confidence limit $\tau \geq 0.55$), the system starts the Text-to-Speech (TTS) tool. Because the `pyttsx3` speech tool cannot safely run multiple tasks at once on Linux or Docker systems, running it directly in the main app window can crash the whole program when the page reloads.

To fix this problem, the app creates a fresh speech engine in a separate, hidden background task for every spoken word. This makes sure the sound plays on its own without freezing the user screen or causing system errors:

```
def speak_letter(text: str) -> None:
    def _speak():
        try:
            import pyttsx3
```

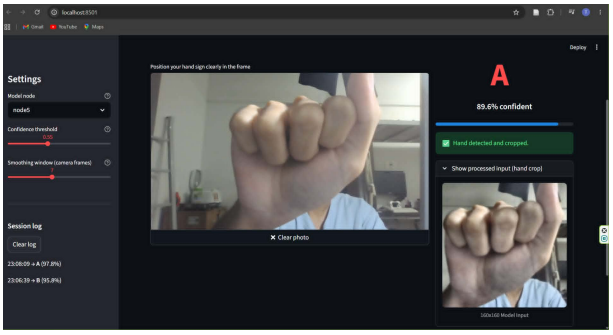

Fig. 7: Real-time Inference Preprocessing: MediaPipe hand extraction, CLAHE lighting normalization, and uniform 160×160 rescaling.

```
            eng = pyttsx3.init()
            eng.setProperty("rate",
150)
            eng.say(f"Letter {text}")
            eng.runAndWait()
            eng.stop()
        except Exception:
            pass
    threading.Thread(target=_speak,
daemon=True).start()
```

This background-task method ensures smooth, continuous sign translation and real-time spoken words, even on small or limited devices.

VII. Conclusions and References

This project successfully designed and implemented a privacy-preserving, decentralized Sign-to-Speech translation system for American Sign Language. By utilizing Decentralized Federated Learning (DFL), the system eliminates the dependency on a centralized cloud server and protects user privacy by keeping sensitive image data local. Five containerized nodes collaborate using a gRPC-based gossip protocol, aggregating local parameters through a weighted FedAvg algorithm. By utilizing a MobileNetV2 backbone and unfreezing the top layers in a dual-phase schedule, the system achieves collaborative validation accuracies of 100% while resolving fine-grained visual differences in hand signs. Real-time inference is demonstrated using a Streamlit UI, MediaPipe hands, convex-hull masking, and Text-to-Speech audio conversion, showcasing a complete and robust distributed AI pipeline.

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial Intelligence and Statistics*, PMLR, 2017, pp. 1273–1282.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[3] C. Zhang, P. Patras, and H. Haddadi, "Deep learning on the edge: A review," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 4, pp. 3217–3242, 2019.

[4] gRPC Authors, "gRPC: A high performance, open source, general RPC framework," *https://grpc.io*, 2026.

[5] F. Chollet et al., "Keras," *https://github.com/fchollet/keras*, 2015.

[6] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2818–2826.

[7] F. P. Kemseu, B. M. R. T. Dong, and F. M. M. T. T. T., "A survey on gossip-based protocols in peer-to-peer networks," *International Journal of Distributed Systems and Technologies*, vol. 12, no. 1, pp. 45–62, 2021.

[8] Streamlit Team, "Streamlit: The fastest way to build and share data apps," *https://streamlit.io*, 2026.